

LESSON 4: SORTING LINEAR DATA

In earlier lessons, we worked with data in many forms: we tracked temperature changes over time, compared students' exam scores with their study hours, and even helped organize a warehouse of electronics and groceries. We learned how to label, clean, and organize data, and how to search through it efficiently using linear and binary search.

But what if you wanted to find the highest temperature, rank students by their exam scores, or organize your inventory from oldest to newest? Before you can do any of that, you need to sort your data.

**SORTING IS THE ARRANGEMENT OF DATA IN A SPECIFIC ORDER
BASED ON A CERTAIN CRITERION.**

Sorting means arranging data in a specific order based on a certain criterion. For example:

- Books can be sorted by their titles.
- Apple crates in a warehouse can be sorted by their arrival date.
- Students can be sorted by their exam scores.

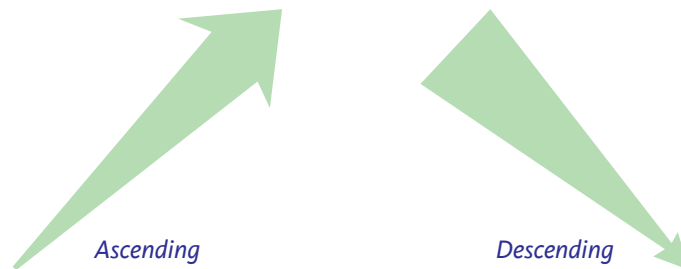
Sorting is a powerful approach to organizing data within any data structure. It typically makes the data more accessible and allows for easier extraction of information.

In this lesson, we will learn what sorting is, why it is useful, and explore two simple but powerful sorting strategies—Insertion Sort and Bubble Sort—and how they work.

ORDER & CRITERIA

Sorting is defined as the arrangement of data in a specific order based on a certain criterion. This definition combines two key ideas: “a specific order” and “a certain criterion.”

Order refers to the direction in which the data is arranged. It answers the question: Which direction will the sorted data follow? The two most common orders are ascending, which means increasing from least to greatest, and descending, which means decreasing from greatest to least.



The criterion, on the other hand, refers to the basis used to determine how the data will be sorted. It answers the question: What are we sorting by? For example, are we sorting by exam scores, temperatures, prices, student names, book titles, or time?

In general, sorting criteria fall into two main categories: numeric data, which consists of numbers, and alphanumeric data, which typically includes letters, words, and characters.

Sorting numeric data is based on numerical value and may be arranged in ascending (e.g. 1, 2, 3...) or descending (e.g. 10, 9, 8...) order. Examples include exam scores, temperatures, prices, and time.

Alphanumeric data is sorted based on character sequences and may also be ordered in ascending (e.g. A, B, C...) or descending (e.g. Z, Y, X...) order. Examples of alphanumeric data include student names, book titles, file names, or ID codes.

Whether a dataset is sorted in a specific order or by a particular criterion depends on the nature of the data and the purpose of sorting.

EXAMPLE 4.1

The school's database contains data on all enrolled students. The table below shows a sample segment of that database with 10 entries. Even though these are just a few data items, it is still difficult and tedious for a person to extract information from it manually.

Name	Family	Absences	Average	Transport
Leila	Carter	1	85	Bus
Tarek	Najem	0	91	Walk
Sofia	Petrovic	10	51	Parents
Ahmed	Dandash	8	64	Bus
Ethan	Müller	0	94	Parents
Dana	Lankart	2	83	Parents
Isabel	Garcia	1	74	Parents
Roland	Bauer	13	45	Walk
Felix	Jensen	3	77	Bus
Ben	Zimmer	0	89	Bus

Let's say we wanted to find a particular family name, "Carter." We would need to check each family name starting from the top — a linear search approach. As we've previously discussed, linear search is not the most efficient algorithm but is necessary when the data is unsorted. However, we can sort this table by family name in ascending alphabetical order.

Name	Family	Absences	Average	Transport
Roland	Bauer	13	45	Walk
Leila	Carter	1	85	Bus
Ahmed	Dandash	8	64	Bus
Isabel	Garcia	1	74	Parents
Felix	Jensen	3	77	Bus
Dana	Lankart	2	83	Parents
Ethan	Müller	0	94	Parents
Tarek	Najem	0	91	Walk
Sofia	Petrovic	10	51	Parents
Ben	Zimmer	0	89	Bus

With a sorted list, finding a name becomes easier. It also allows us to apply more efficient searching algorithms, such as the binary search algorithm.

The same dataset can be sorted using different orders and rules. The choice of order and rule depends on the nature of the data and the purpose of sorting.

EXAMPLE 4.2

Let's say, for example, that this time we want to extract information about absences and attendance. If our purpose is to identify the students who were absent the greatest number of times, and the data under "Absences" contains numeric values, then we would sort the dataset by Absences in descending order of numeric value. With the data now sorted, identifying the students with the highest number of absences becomes much easier.

Name	Family	Absences	Average	Transport
Roland	Bauer	13	45	Walk
Sofia	Petrovic	10	51	Parents
Ahmed	Dandash	8	64	Bus
Felix	Jensen	3	77	Bus
Dana	Lankart	2	83	Parents
Leila	Carter	1	85	Bus
Isabel	Garcia	1	74	Parents
Tarek	Najem	0	91	Walk
Ethan	Müller	0	94	Parents
Ben	Zimmer	0	89	Bus

Not only does sorting make finding information easier—either by making it visually evident or by enabling the use of efficient search algorithms—but it also helps us spot trends and patterns in data, particularly in smaller datasets like this one.

One of the key benefits of sorting is that it makes patterns more visible. When data is arranged in a meaningful order—such as alphabetically or numerically—it becomes easier to scan, compare, and interpret. For example, sorting students by the number of absences in descending order quickly highlights those with the highest absenteeism, drawing attention to potential attendance issues.

Similarly, sorting exam scores from highest to lowest reveals the top performers in a class. But it can also help uncover broader trends.

EXAMPLE 4.3

If we examine the table—specifically the fields Absences and Average—we notice a pattern: students with the most absences tend to have the lowest averages, while those with the fewest absences tend to have the highest.

Name	Family	Absences	Average	Transport
Roland	Bauer	13	45	Walk
Sofia	Petrovic	10	51	Parents
Ahmed	Dandash	8	64	Bus
Felix	Jensen	3	77	Bus
Dana	Lankart	2	83	Parents
Leila	Carter	1	85	Bus
Isabel	Garcia	1	74	Parents
Tarek	Najem	0	91	Walk
Ethan	Müller	0	94	Parents
Ben	Zimmer	0	89	Bus

It is important to note that, although this observation does not confirm a relationship between attendance and academic performance, it raises a question that could be explored through further analysis.

As we've seen, sorting is a valuable strategy for organizing data in a way that makes it easier to search, interpret, and extract information. Whether we're searching for a specific name or analyzing attendance trends, sorting helps reveal patterns and supports more efficient decision-making. While sorting alone does not confirm relationships within data, it can highlight areas worth investigating.

The same dataset can be sorted in different ways depending on what we're looking for and why. Sorting by alphabetical order helps us locate items more quickly, while sorting numerically allows us to rank or compare values. As we continue exploring sorting strategies, we'll begin to see how even simple algorithms like Insertion Sort and Bubble Sort can support meaningful data analysis when applied thoughtfully.

SORTING ALGORITHMS

Sorting algorithms determine the step-by-step process of arranging data in a specific order. All sorting algorithms must be able to compare data items and change their positions so that the data becomes organized. While the goal is the same, different algorithms follow different strategies—each varying in which data items are compared, and how and when those items are moved to new positions.

In this section, we will explore two simple but effective sorting algorithms—Insertion Sort and Bubble Sort.

THE INSERTION SORT ALGORITHM

The insertion sort algorithm follows a strategy of compare, shift, and insert. Starting at the beginning of the list, it temporarily removes one data item at a time and compares it with the items that precede it. Once the correct position is found, any intervening out-of-order data items are shifted up, and the removed data item is inserted into the empty position closer to the beginning.

THE INSERTION SORT ALGORITHM FOLLOWS A SEQUENTIAL STRATEGY OF COMPARE, SHIFT, AND INSERT.

In a sense, out-of-order items are shifted toward the end, and data items are inserted one by one into their correct positions.

If the goal is to sort in ascending order, data items with lesser values are inserted in increasing order near the beginning of the list, while greater out-of-order values are shifted upward toward the end. If the goal is to sort in descending order, the opposite occurs: greater values are inserted in decreasing order near the beginning, while lesser out-of-order values are shifted upward.

EXAMPLE 4.4

In this example, we will sort a list of five numbers in ascending order. Starting at index 1, each data item is compared to the values before it. Greater out-of-order values are shifted upward, and the lesser value is inserted into its correct position near the beginning of the list. These steps are repeated in iterative loops until the last item in the list has been selected, compared, and inserted into its correct position.

“5, 3, 4, 1, 2”

To better visualize the list, each data item is shown in its respective position, labeled by its index.

DATA	5	3	4	1	2
INDEX	0	1	2	3	4

The First Iteration

Step 1 - Hold

Data item at Index 1 is temporarily held.

		3			
DATA	5		4	1	2
INDEX	0	1	2	3	4

Step 2 - Compare

Yes, “5” is greater than “3”.

		3			
DATA	5		4	1	2
INDEX	0	1	2	3	4

Step 3 - Shift

“5” is shifted to Index 1 instead of “3”.

		3			
DATA		5	4	1	2
INDEX	0	1	2	3	4

Step 4 - Insert

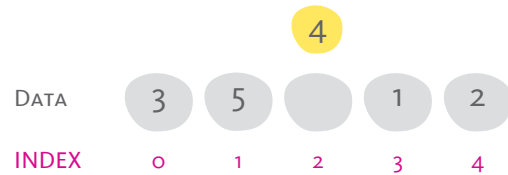
“3” is inserted at Index 0 instead of “5”.

DATA	3	5	4	1	2
INDEX	0	1	2	3	4

The Second Iteration

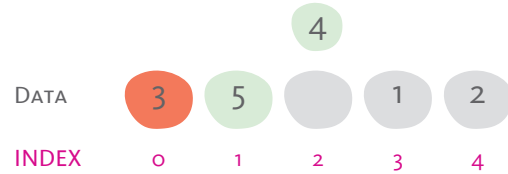
Step 1 - Hold

Data item at Index 2 is temporarily held.

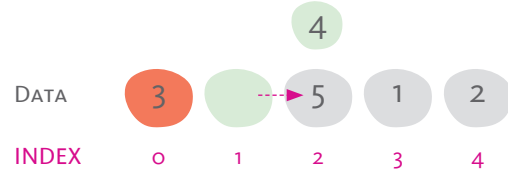
**Step 2 - Compare**

Yes, "5" is greater than "4".

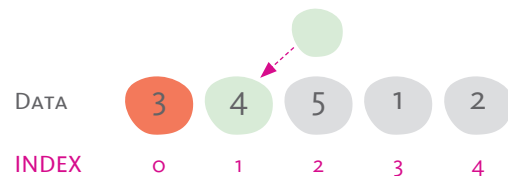
No, "3" is not greater than "4".

**Step 3 - Shift**

Only "5" is shifted to Index 2 instead of "4".

**Step 4 - Insert**

"4" is inserted at Index 1 instead of "5".



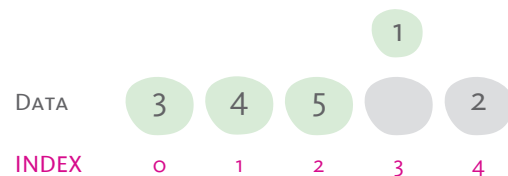
The Third Iteration

Step 1 - Hold

Data item at Index 3 is temporarily held.

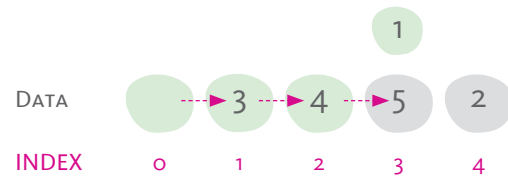
**Step 2 - Compare**

Yes, "5", "4", and "3" are greater than "1".



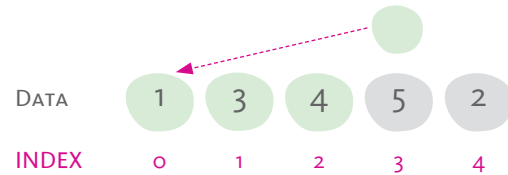
Step 3 - Shift

"5", "4", and "3" are shifted one index up.



Step 4 - Insert

"1" is inserted at Index 0 instead of "3".



The Fourth Iteration

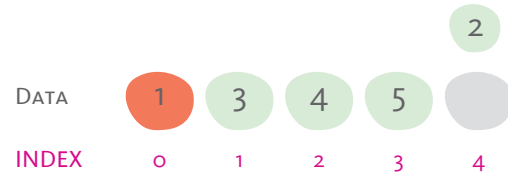
Step 1 - Hold

Data item at Index 4 is temporarily held.



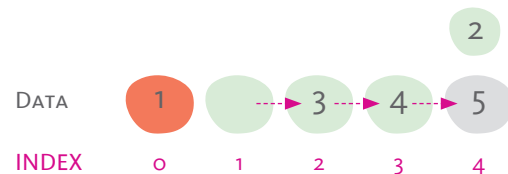
Step 2 - Compare

Yes, "5", "4", and "3" are greater than "2".
No, "1" is not greater than "2".



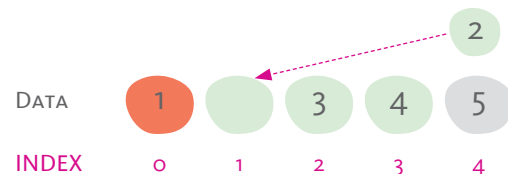
Step 3 - Shift

"5", "4", and "3" are shifted one index up.



Step 4 - Insert

"2" is inserted at Index 1 instead of "3".



Sorting Complete

The last index is reached.

All data is sorted in ascending order.



THE INSERTION SORT PROCESS

Now that we have demonstrated the mechanics of the algorithm, we need to establish the flow of steps that define the operations and conditions that make the insertion sort algorithm work.

VARIABLES & LABELS

Understanding how sorting works begins with recognizing how we name and refer to both data and positions within a dataset. In programming, we use variables as labels that help us hold and manipulate information.

A **VARIABLE** IS A LABEL WHOSE VALUE OR CONTENT CAN CHANGE.

In the insertion sort process, we use several key variables to make the algorithm readable and workable. These variables may represent either the data values themselves or their positions (called indices) within the list:

- **hold_index** refers to the index whose data is temporarily held for comparison.
- **hold_data** refers to the data at the hold_index.
- **compare_index** refers to the index that precedes the hold_index and against which the held data is compared.
- **compare_data** refers to the data at the compare_index.

It is important to remember the difference between data and index:

- Indices are fixed positions in the list—they do not change.
- Data can move from one index to another during sorting.

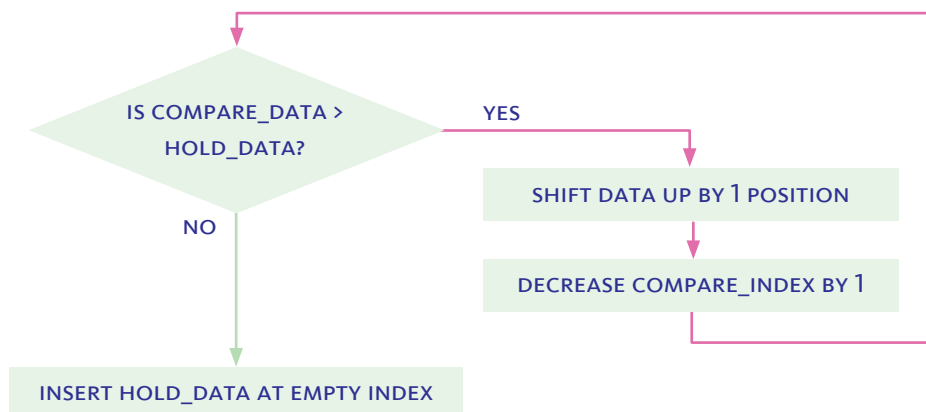
FLOW & CONTROL

Every algorithm has a flow or sequence of steps that are executed in order to achieve the required outcome—in this case, sorting a list of numbers in ascending order. This flow is controlled by certain conditions that determine the sequence in which instructions are carried out.

When a group of instructions needs to be repeated, they are collectively referred to as a loop. The insertion sort algorithm can generally be described as having two repeating sets of steps, which we will refer to as the outer loop and the inner loop.

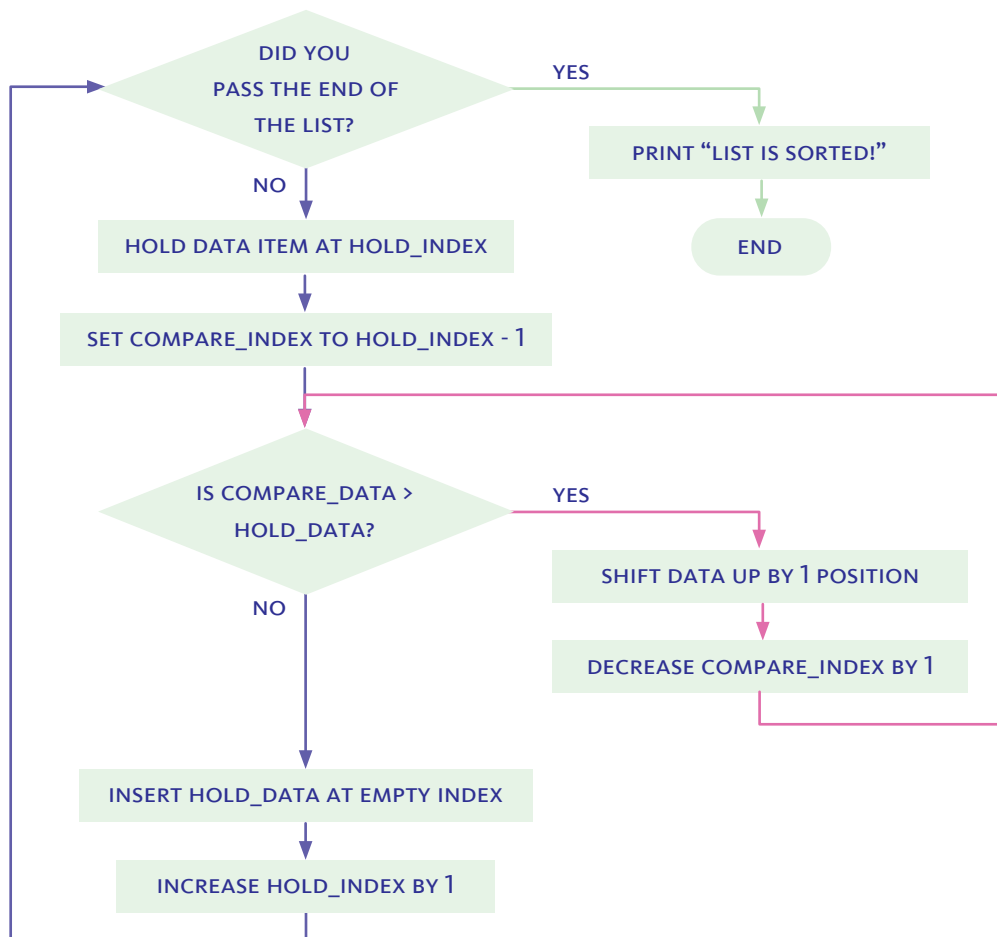
The **inner loop** performs a series of steps that compare the held data item with the items before it in the list. If the compared data is greater than the held data, it is shifted forward by one position. This process repeats as long as that condition is true.

Once the compared data is less than or equal to the held data, the loop ends, and the algorithm inserts the held data into the correct empty position toward the beginning of the list.



The inner loop compares and shifts data, moving one position at a time toward the beginning of the list. With each step, it shifts larger data items up by one index. The loop breaks when the compared data is less than or equal to the held data, at which point the held data is inserted into the empty index created by the shifting.

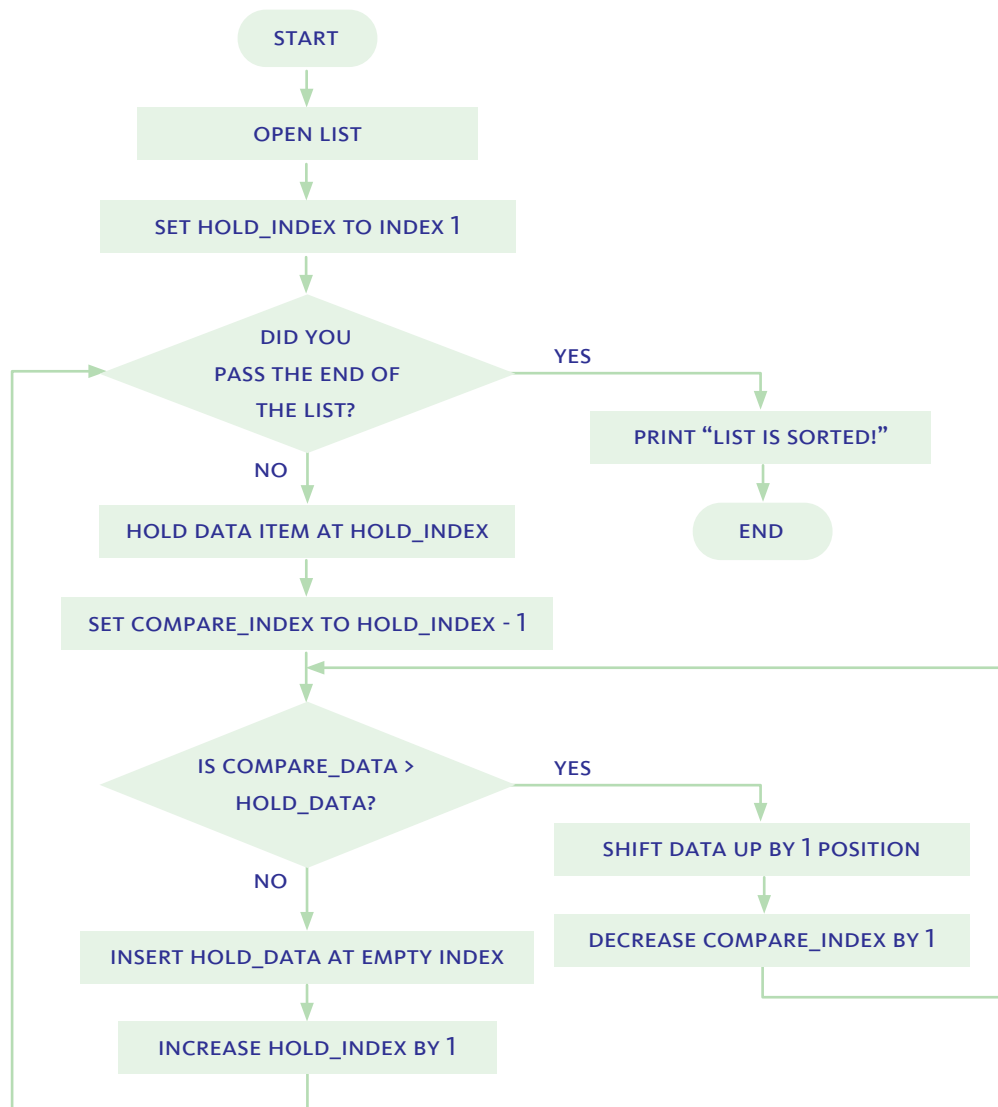
The **outer loop** is tasked with moving the inner loop sequentially through each index in the list. It starts at index 1, and at each step, it passes control to the inner loop to perform the comparisons, shifts, and insertion. When the inner loop completes its task, control is handed back to the outer loop, which moves the process forward by one index. The outer loop breaks when its condition is satisfied—that is, when it reaches the end of the list.



This flowchart highlights the two main components of the insertion sort process. The outer loop (blue) moves sequentially through each data item in the list using the `hold_index`. The inner loop (pink) repeatedly compares the held data with preceding items and shifts larger values upward until the correct position is found for insertion.

INITIALIZING

Initialization is the first part of the algorithm where we get everything ready before sorting begins. In this phase, we open the list we want to sort and set the starting point by choosing the first index we'll work with.



This flowchart shows the step-by-step process of the insertion sort algorithm. It begins with initialization, then uses an outer loop to move through the list and an inner loop to compare and shift data until each item is inserted into its correct position.

THE INSERTION SORT PROGRAM

Having understood the mechanics of the insertion sort algorithm and the flow of its process, it is now time to write the instructions for a program that implements this algorithm.

The program is divided into four blocks:

1. **Initialization** — this is where we begin by opening the list and declaring the variables needed for the sorting process. Before a variable can be used, it must first be declared and given an initial value. In this case, we declare `hold_index` and assign it the value 1, so that sorting starts with the second item in the list. We also declare `end_index` and assign it the value of the last index in the list, which sets the boundary for when the sorting process should stop.



```
START InsertionSort
OPEN list
SET hold_index (1)
SET end_index (last index of list)

REPEAT_UNTIL hold_index > end_index
  SET hold_data (data_item AT hold_index)
  SET compare_index (hold_index - 1)
  SET compare_data (data_item AT compare_index)

  AS_LONG_AS hold_data > compare_data
    SHIFT compare_data FROM compare_index TO compare_index + 1
    RESET compare_index (compare_index - 1)
    RESET compare_data (data_item AT compare_index)

  INSERT hold_data TO compare_index
  RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END InsertionSort
```

The pink brackets and green text indicate the initialization block of the insertion sort algorithm. This section sets up the list and defines key variables—`hold_index` and `end_index`—to prepare the algorithm before the sorting process begins.

- 2. Outer loop** — which moves one step at a time through the list, selecting the next data item to be sorted.

This loop is controlled by a **REPEAT_UNTIL** instruction, which allows the loop to continue until it reaches the end of the list—i.e., when the `hold_index` is greater than the `end_index`.

Inside this block, we initialize the `hold_data` to be the data item at `hold_index`, the `compare_index` to be one position before the `hold_index`, and the `compare_data` to be the value at the `compare_index`.

The block concludes with inserting the held data into the appropriate position, as determined by the inner loop, and then incrementing `hold_index` by 1 to prepare for the next outer loop cycle.

```
START InsertionSort
OPEN list
SET hold_index (1)
SET end_index (last index of list)

REPEAT_UNTIL hold_index > end_index
  SET hold_data (data_item AT hold_index)
  SET compare_index (hold_index - 1)
  SET compare_data (data_item AT compare_index)

  AS_LONG_AS hold_data > compare_data
    SHIFT compare_data FROM compare_index TO compare_index + 1
    RESET compare_index (compare_index - 1)
    RESET compare_data (data_item AT compare_index)

  INSERT hold_data TO compare_index
  RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END InsertionSort
```

The pink brackets and green text indicate the outer loop of the algorithm. This loop controls the progression through the list, selecting one data item at a time using the `hold_index`. For each pass, it prepares the held data and sets up comparisons before calling the inner loop. The loop continues until the `hold_index` has passed the `end_index`, meaning the entire list has been sorted.

3. **Inner loop** — which compares the held data with the values before it, shifts out-of-order data up the list, and determines where the held data should be inserted.

This loop uses an **AS_LONG_AS** instruction, which means that as long as the value of `hold_data` is greater than the value of `compare_data`, the algorithm continues shifting the `compare_data` up by one position and resetting `compare_index` one step back.

```
START InsertionSort
OPEN list
SET hold_index (1)
SET end_index (last index of list)

REPEAT_UNTIL hold_index > end_index
  SET hold_data (data_item AT hold_index)
  SET compare_index (hold_index - 1)
  SET compare_data (data_item AT compare_index)

  AS_LONG_AS hold_data > compare_data
    SHIFT compare_data FROM compare_index TO compare_index + 1
    RESET compare_index (compare_index - 1)
    RESET compare_data (data_item AT compare_index)

  INSERT hold_data TO compare_index
  RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END InsertionSort
```

The pink brackets and green text indicate the inner loop, which repeatedly compares the held data with earlier items in the list. As long as the held data is smaller, larger values are shifted upward one position. The loop continues until the correct position is found for inserting the held data.

Notes

Program blocks can be arranged:

- Sequentially – one after another in order.
- Nested – with one block contained inside another.

- 4. Termination** — After all data items have been compared, shifted, and inserted into their correct positions, the program reaches its termination block.

This part is not part of any loop. It simply prints a message to indicate that the sorting is complete and then ends the program. It serves as a clear signal that the algorithm has finished running.

```
START InsertionSort
OPEN list
SET hold_index (1)
SET end_index (last index of list)

REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    AS_LONG_AS hold_data > compare_data
        SHIFT compare_data FROM compare_index TO compare_index + 1
        RESET compare_index (compare_index - 1)
        RESET compare_data (data_item AT compare_index)

    INSERT hold_data TO compare_index
    RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END InsertionSort
```

The pink brackets and green text indicate the termination block of the program. This block runs after the sorting process is complete. It prints a message to confirm that the list has been successfully sorted and then ends the program.

Notes

Indentation in a program shows which instructions belong to a block. Lines indented under a loop or condition are part of that block. Further indentation indicates nested blocks.

THE BUBBLE SORT ALGORITHM

The bubble sort algorithm follows a strategy of pair, compare, and swap. Starting at the beginning of the list, it repeatedly compares pairs of adjacent data items. If the two items are out of order, they are swapped so that the values begin to move toward their correct positions.

THE BUBBLE SORT ALGORITHM FOLLOWS A SEQUENTIAL STRATEGY OF PAIR, COMPARE, AND SWAP.

If the goal is to sort in ascending order, greater values are bubbled toward the end of the list with each full pass, while smaller values gradually settle into place near the beginning. This process is repeated through multiple passes until the entire list is sorted.

If the goal is to sort in descending order, each pass pushes the smallest unsorted value toward the end, while the greater values settle into place near the beginning.

EXAMPLE 4.5

Let's go back to the same list we used in Example 4.4, and this time use the bubble sort algorithm to sort its data in ascending order.

"5, 3, 4, 1, 2"

The bubble sort algorithm makes several passes through the list, each time sequentially comparing pairs of data items starting from the beginning. Like the insertion sort algorithm, bubble sort uses two nested loops. The outer loop controls the number of passes through the list, with each pass considered an iteration of the loop. The inner loop handles the individual pairing, comparing, and swapping of data items, and it iterates through each pair in the list.

The First Iteration

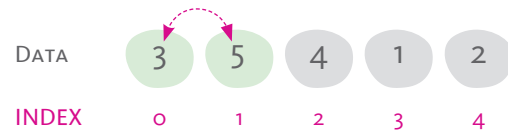
Step 1 - Pair & Compare

Index 0 and Index 1 are paired.

Yes, "5" is greater than "3"

Step 2 - Swap

"5" and "3" are swapped.

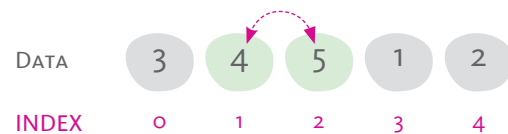
**Step 1 - Pair & Compare**

Index 1 and Index 2 are paired.

Yes, "5" is greater than "4"

Step 2 - Swap

"5" and "4" are swapped.

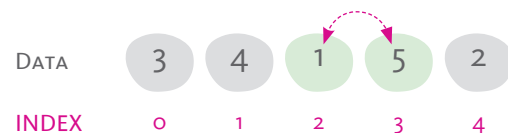
**Step 1 - Pair & Compare**

Index 2 and Index 3 are paired.

Yes, "5" is greater than "1"

Step 2 - Swap

"5" and "1" are swapped.

**Step 1 - Pair & Compare**

Index 3 and Index 4 are paired.

Yes, "5" is greater than "2"

Step 2 - Swap

"5" and "2" are swapped.



The Second Iteration

Step 1 - Pair & Compare

Index 0 and Index 1 are paired.

No, "3" is not greater than "4"

DATA	3	4	1	2	5
INDEX	0	1	2	3	4

Step 2 - Swap

"3" and "4" are not swapped.

DATA	3	4	1	2	5
INDEX	0	1	2	3	4

Step 1 - Pair & Compare

Index 1 and Index 2 are paired.

Yes, "4" is greater than "1"

DATA	3	4	1	2	5
INDEX	0	1	2	3	4

Step 2 - Swap

"4" and "1" are swapped.

DATA	3	1	4	2	5
INDEX	0	1	2	3	4

Step 1 - Pair & Compare

Index 2 and Index 3 are paired.

Yes, "4" is greater than "2"

DATA	3	1	4	2	5
INDEX	0	1	2	3	4

Step 2 - Swap

"4" and "2" are swapped.

DATA	3	1	2	4	5
INDEX	0	1	2	3	4

Step 1 - Pair & Compare

Index 3 and Index 4 are paired.

Yes, "4" is not greater than "5"

DATA	3	1	2	4	5
INDEX	0	1	2	3	4

Step 2 - Swap

"4" and "5" are not swapped.

DATA	3	1	2	4	5
INDEX	0	1	2	3	4

The Third Iteration

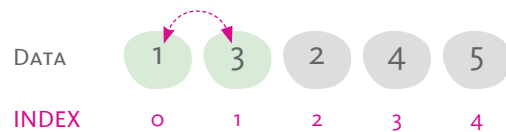
Step 1 - Pair & Compare

Index 0 and Index 1 are paired.

Yes, "3" is greater than "1"

**Step 2 - Swap**

"3" and "1" are swapped.

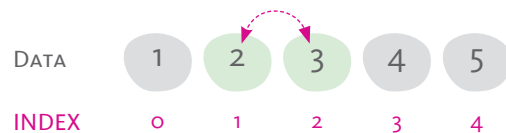
**Step 1 - Pair & Compare**

Index 1 and Index 2 are paired.

Yes, "3" is greater than "2"

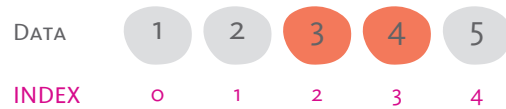
**Step 2 - Swap**

"3" and "2" are swapped.

**Step 1 - Pair & Compare**

Index 2 and Index 3 are paired.

No, "3" is not greater than "4"

**Step 2 - Swap**

"3" and "4" are not swapped.

**Step 1 - Pair & Compare**

Index 3 and Index 4 are paired.

No, "4" is not greater than "5"

**Step 2 - Swap**

"4" and "5" are not swapped.



The Fourth Iteration

Step 1 - Pair & Compare

Index 0 and Index 1 are paired.
No, "1" is not greater than "2"

DATA	1	2	3	4	5
INDEX	0	1	2	3	4

Step 2 - Swap

"1" and "2" are not swapped.

DATA	1	2	3	4	5
INDEX	0	1	2	3	4

Step 1 - Pair & Compare

Index 1 and Index 2 are paired.
No, "2" is not greater than "3"

DATA	1	2	3	4	5
INDEX	0	1	2	3	4

Step 2 - Swap

"2" and "3" are not swapped.

DATA	1	2	3	4	5
INDEX	0	1	2	3	4

Step 1 - Pair & Compare

Index 2 and Index 3 are paired.
No, "3" is not greater than "4"

DATA	1	2	3	4	5
INDEX	0	1	2	3	4

Step 2 - Swap

"3" and "4" are not swapped.

DATA	1	2	3	4	5
INDEX	0	1	2	3	4

Step 1 - Pair & Compare

Index 3 and Index 4 are paired.
No, "4" is not greater than "5"

DATA	1	2	3	4	5
INDEX	0	1	2	3	4

Step 2 - Swap

"4" and "5" are not swapped.

DATA	1	2	3	4	5
INDEX	0	1	2	3	4

THE BUBBLE SORT PROCESS

Having understood the mechanics and demonstrated the algorithm step by step, we can now take a closer look at the underlying operations and conditions that make the bubble sort process work. By identifying these key components, we can begin to organize the logic into clear instructions and structured loops that reflect how the algorithm actually functions.

DATA & INDICES

As in any algorithm, we must start by defining our variables. These variables must be declared before they are used, and each is assigned an initial value that may change as the algorithm runs.

- **hold_index** refers to the index whose data is temporarily held for comparison.
- **hold_data** refers to the data at the hold_index.
- **compare_index** refers to the index that precedes the hold_index, and against which the held data is compared.
- **compare_data** refers to the data at the compare_index.
- **end_index** refers to the last index in the list. In this case, it remains fixed throughout the sorting process.

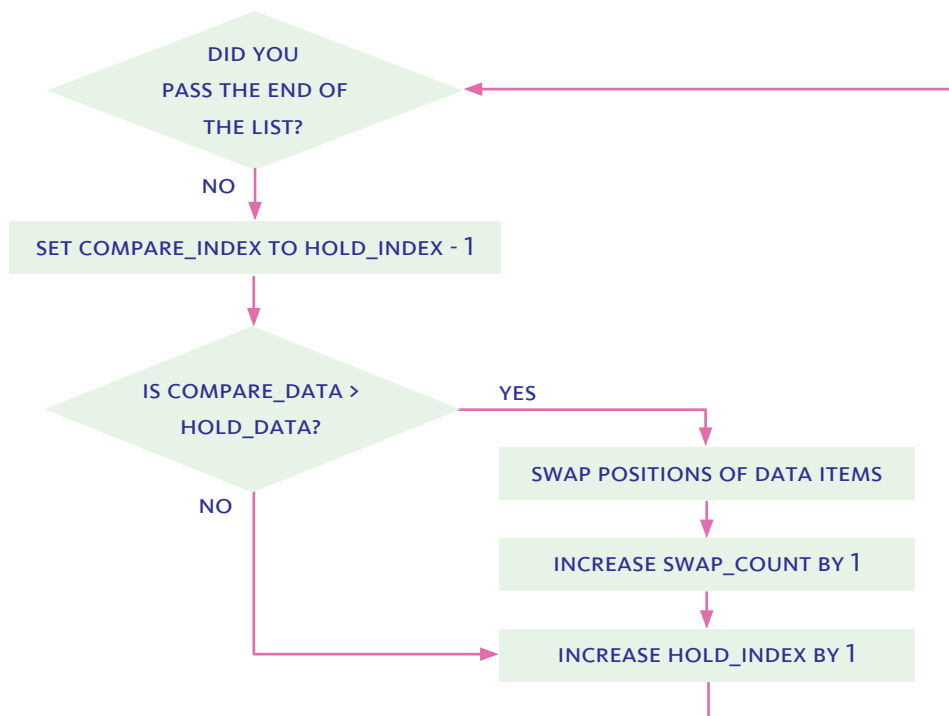
In addition to these variables—previously encountered in the insertion sort algorithm—the bubble sort algorithm requires an additional variable:

- **swap_count** is a variable that keeps track of the number of times a swap occurs during a single pass through the list.

LOOPS & CONDITIONS

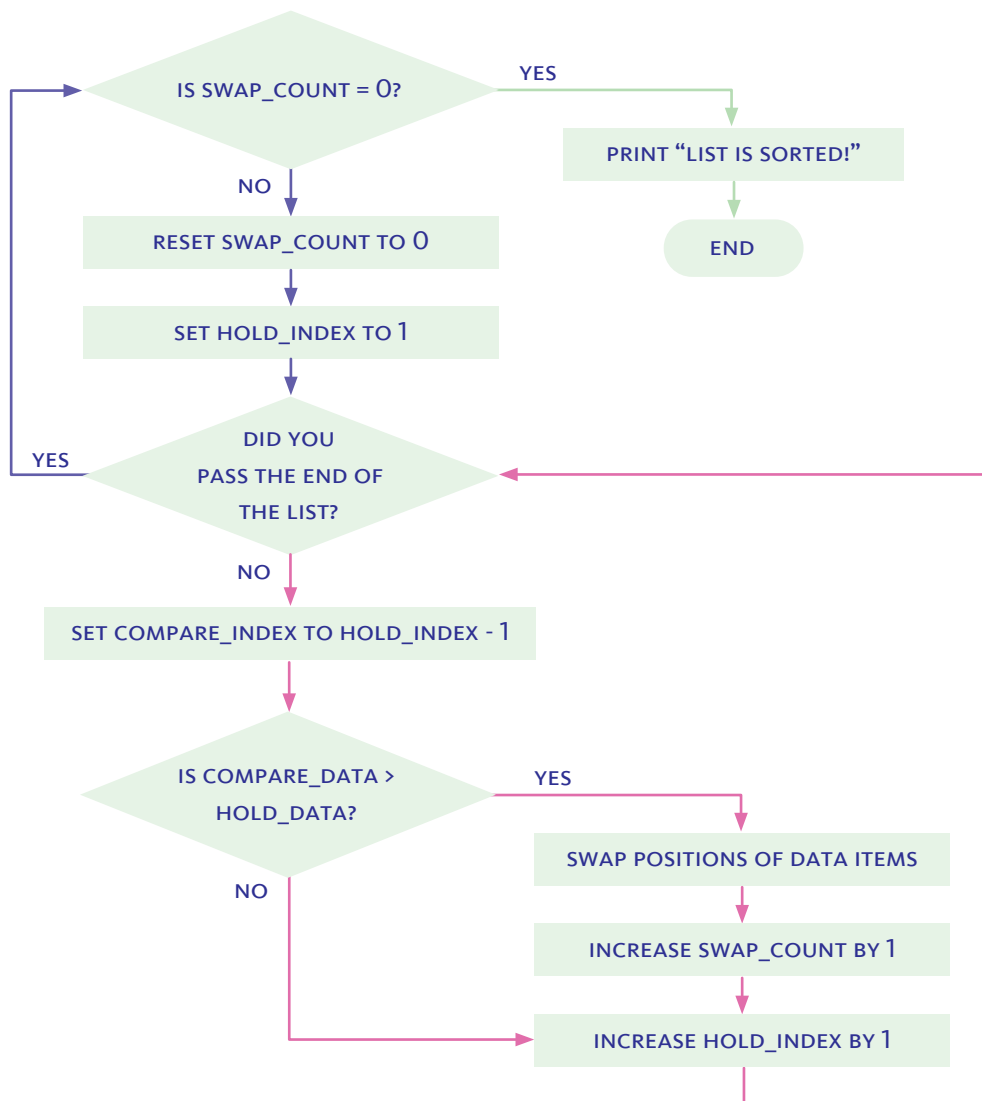
The bubble sort algorithm uses two loops: an inner loop and an outer loop. The inner loop performs a single pass through the list, pairing, comparing, and swapping adjacent data items. The outer loop repeats these passes until no swaps are needed, meaning the list is fully sorted.

The **inner loop** performs a series of steps that compare the held data item with the items before it in the list. If the compared data is greater than the held data, it is shifted forward by one position. This process repeats as long as that condition is true. Once the compared data is less than or equal to the held data, the condition is no longer satisfied. At that point, the loop ends, and the algorithm inserts the held data into the correct empty position toward the beginning of the list.



This flowchart illustrates the inner loop of the bubble sort process. It begins by setting the `compare_index` to one position before the `hold_index`, forming a pair of adjacent data items. The algorithm compares their values and swaps them if needed. Each time a swap occurs, the `swap_count` is increased. The `hold_index` then moves forward, and the loop continues until the end of the list is reached, completing a full pass.

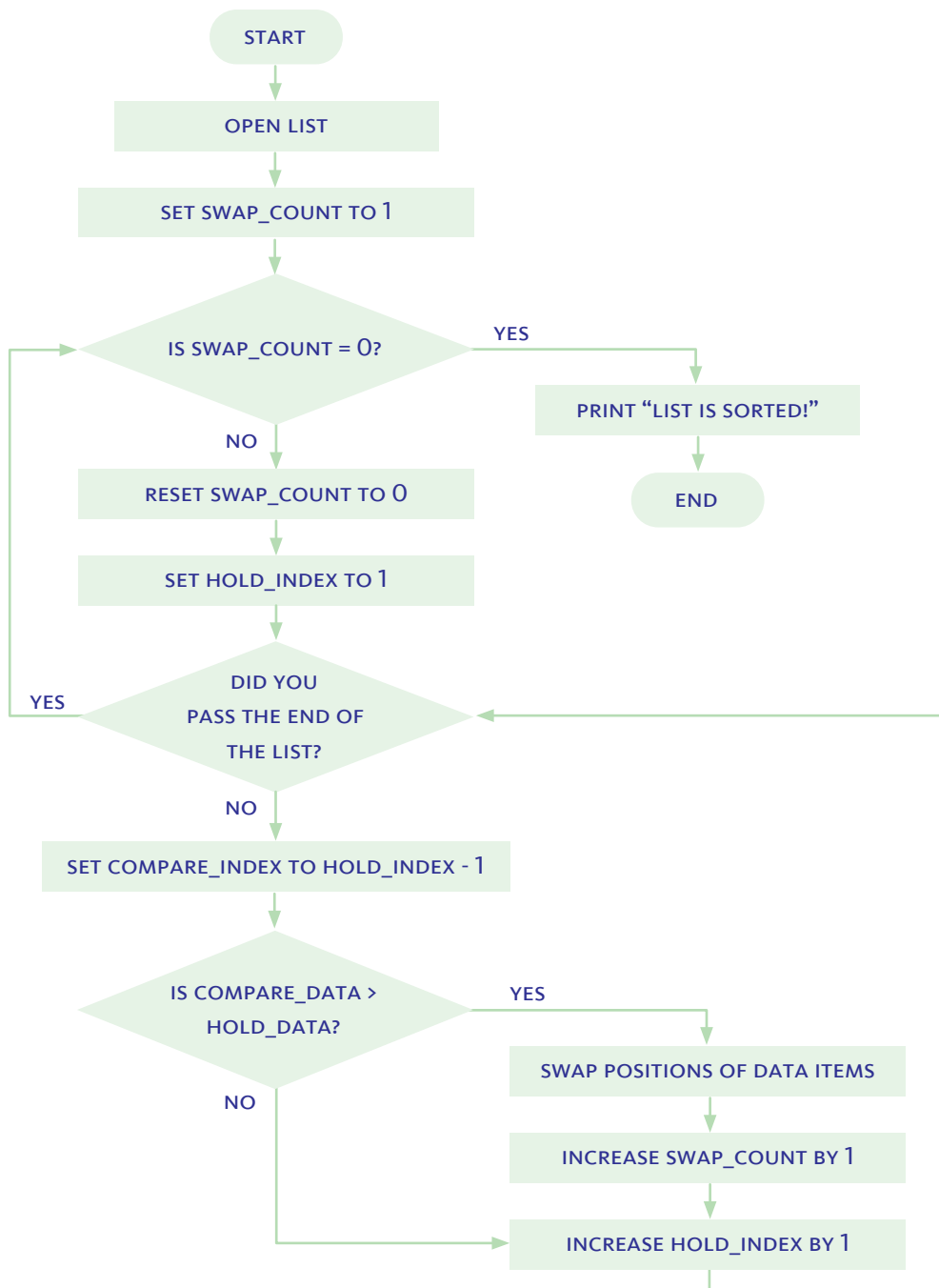
The **outer loop** is tasked with repeating the passes until the list is fully sorted. After each pass, it checks `swap_count`. If `swap_count` is 0, no swaps occurred, and the list is sorted. If not, the list is not completely sorted. The loop prepares for another pass by resetting `swap_count` to 0, moving `hold_index` back to the start, and running the pass again. This continues until a pass completes with no swaps.



The outer loop (blue) repeats passes through the list until no swaps occur. The inner loop (pink) performs a single pass, comparing and swapping adjacent data items as needed.

INITIALIZING

First, the list is opened, and the variable `swap_count` is set to 1. Since the loop stops when `swap_count` equals 0—meaning no swaps were made—we cannot start with it set to 0, or the algorithm would end before it begins. Setting it to 1 ensures that the sorting process is able to start.



THE BUBBLE SORT PROGRAM

Now that we understand how the bubble sort algorithm works and how its steps flow, we can begin writing the instructions for a program that implements it.

The program has four blocks:

1. **Initialization** — This is where we begin by opening the list and declaring the variables needed for the sorting process. Before a variable can be used, it must first be declared and given an initial value. In this case, we declare `end_index` and assign it the value of the last index in the list, setting the boundary for the sorting range. We also declare `swap_count` and assign it an initial value of 1 to ensure that the algorithm enters the outer loop for its first pass.



```
START BubbleSort
OPEN list
SET end_index (last index of list)
SET swap_count (1)

REPEAT_UNTIL swap_count (0)
  RESET swap_count (0)
  SET hold_index (1)

  REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    IF compare_data > hold_data THEN
      SWAP_POSITIONS (compare_data, hold_data)
      RESET swap_count (swap_count + 1)

    RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END BubbleSort
```

The pink brackets and green text indicate the initialization block of the bubble sort algorithm. These variables are essential to prepare the algorithm before the sorting process starts.

2. Outer loop — The outer loop controls the overall sorting process by repeating passes through the list until it is fully sorted.

It begins with a **REPEAT_UNTIL** instruction that continues looping until `swap_count` is 0, indicating no swaps occurred during the last pass. At the start of each pass, the algorithm resets `swap_count` to 0 and sets `hold_index` to 1, preparing to step through the list. This outer loop ensures that the sorting continues as long as necessary to bring all elements into order.

```
START BubbleSort
OPEN list
SET end_index (last index of list)
SET swap_count (1)

REPEAT_UNTIL swap_count (0)
  RESET swap_count (0)
  SET hold_index (1)

  REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    IF compare_data > hold_data THEN
      SWAP_POSITIONS (compare_data, hold_data)
      RESET swap_count (swap_count + 1)

  RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END BubbleSort
```

The pink bracket and green-highlighted text indicate the outer loop block of the bubble sort algorithm. This section controls how many passes the algorithm makes through the list, repeating the process until no swaps occur—signaling that the list is sorted.

- 3. Inner loop** — is responsible for performing a single pass through the list, comparing and swapping adjacent items if needed.

It begins with a **REPEAT_UNTIL** instruction that runs as long as `hold_index` is less than or equal to `end_index`. At each step, the algorithm sets `hold_data` to the value at `hold_index`, identifies `compare_index` as the position just before it, and retrieves `compare_data` from that position. If `compare_data` is greater than `hold_data`, the two values are swapped, and `swap_count` is increased by one. The `hold_index` is then incremented, and the loop continues through the rest of the list.

```
START BubbleSort
OPEN list
SET end_index (last index of list)
SET swap_count (1)

REPEAT_UNTIL swap_count (0)
  RESET swap_count (0)
  SET hold_index (1)

  REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    IF compare_data > hold_data THEN
      SWAP_POSITIONS (compare_data, hold_data)
      RESET swap_count (swap_count + 1)

    RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END BubbleSort
```

The pink bracket and green-highlighted text indicate the inner loop block of the bubble sort algorithm. This section performs comparisons and swaps between adjacent items as it moves through the list from left to right during a single pass.

- 4. Termination** — Once the sorting process is complete—meaning a full pass has been made with no swaps—the algorithm exits the outer loop. At this point, it prints the message “List is sorted!” to confirm that the list has been successfully ordered. The program then ends.

```
START BubbleSort
OPEN list
SET end_index (last index of list)
SET swap_count (1)

REPEAT_UNTIL swap_count (0)
  RESET swap_count (0)
  SET hold_index (1)

  REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    IF compare_data > hold_data THEN
      SWAP_POSITIONS (compare_data, hold_data)
      RESET swap_count (swap_count + 1)

  RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END BubbleSort
```

The pink bracket and green-highlighted text indicate the termination block of the bubble sort algorithm. This section confirms that the list is sorted and marks the end of the program.

Notes

AS_LONG_AS and REPEAT_UNTIL resemble natural language, making them ideal for beginners. They help students focus on how conditions control the flow of a program and highlight the key distinction between pre-check and post-check logic:

- AS_LONG_AS runs while a condition remains true.
- REPEAT_UNTIL runs until a condition becomes true.

PERFORMANCE

When more than one algorithm can solve the same problem, performance becomes an important way to compare them. Two measures of performance are **step count** and **execution time**.

STEP COUNT AND EXECUTION TIME ARE TWO MEASURES OF PERFORMANCE.

Step count refers to the number of steps required by the algorithm to complete the task—which, in this case, is sorting. This measure helps us understand the structure and logical efficiency of an algorithm. Fewer steps means the algorithm performs a smaller number of operations to complete its task.

Execution time refers to how long it takes for the program to run and sort a particular dataset. This measure tells us about performance in practice. Shorter execution time means the program finishes faster—but that also depends on how long each type of step takes.

Both measures are useful—and together, they give us a fuller picture of performance.

WORST-CASE SCENARIO

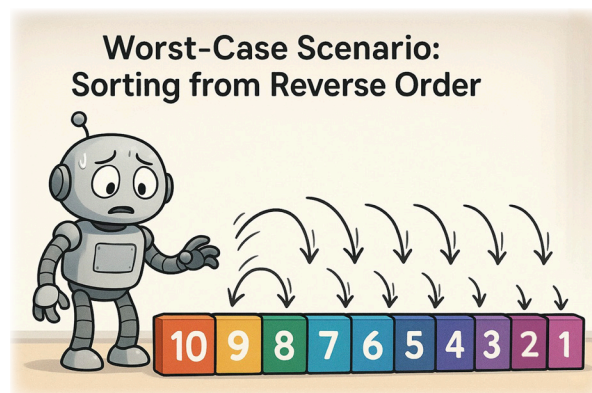
Algorithms and programs can perform differently depending on the dataset they are given. Since datasets can vary considerably, we typically use the worst-case scenario as a standard way to evaluate performance.

In the case of sorting, the worst-case scenario occurs when the list is arranged in the exact opposite order of the desired outcome. So, if we are trying to sort a list in ascending order, the worst case would be a list that starts in descending order. In this situation, the algorithm must shift or swap nearly every item to put the list in the correct order.

We use the worst case because:

1. It shows us the maximum number of steps an algorithm might take—this helps us understand its performance limits.
2. It prepares us for real-world situations where the data might be in the least helpful or most disorganized state.
3. It allows us to fairly compare algorithms by seeing how they handle the same challenging scenario.

In other words, if an algorithm performs well in the worst case, we can be confident that it will perform even better when the data is easier to work with.



INSERTION SORT

Let's explore the performance of insertion sort under the worst-case scenario: a list that is completely reversed.

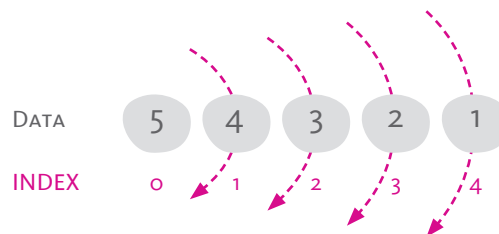
We will assume a list of 5 numbers that need to be sorted in ascending order. Since we want the worst-case, the list will start in descending order, meaning every number is in the opposite position of where it should be.

DATA	5	4	3	2	1
INDEX	0	1	2	3	4

STEP COUNT

When we examine the steps, we'll group them into two categories: **comparisons** and **moves**. The difference is that comparisons can be done with data in place, while moves involve the physical repositioning of data—either through a hold, shift, or insert.

The algorithm starts at index 1 and moves through the list one index at a time until the end. This process is controlled by the outer loop. For a list with 5 data items, the outer loop will run 4 times, and during each iteration, it will hold one item for sorting.



Each curved arrow represents one iteration of the outer loop in the insertion sort algorithm. At each index, the algorithm holds the current value and prepares to compare it with the values in all preceding indices. This process repeats from index 1 to the end of the list.

Now let's look at the inner loop. At each index, the inner loop compares the held data with the data at all preceding indices. Since this is a worst-case scenario, every comparison results in a shift.

- At index 1, there is 1 comparison and 1 shift.
- At index 2, there are 2 comparisons and 2 shifts.
- At index 3, there are 3 comparisons and 3 shifts.
- At index 4, there are 4 comparisons and 4 shifts.

Since this is a worst-case scenario, every comparison results in a shift. That means the number of shifts is equal to the number of comparisons, which again equals the number of preceding indices.

Finally, we are left with the insertions. These occur once during each iteration of the outer loop, giving us 4 insertions in total.

Index	Hold	Compare	Shift	Insert
1	1	1	1	1
2	1	2	2	1
3	1	3	3	1
4	1	4	4	1
Total	4	10	10	4

So for a 5-item dataset, under the worst-case scenario of a completely reversed list, the insertion sort algorithm requires a total step count of 4 (holds) + 10 (comparisons) + 10 (shifts) + 4 (insertions) = 28 steps.

EXECUTION TIME

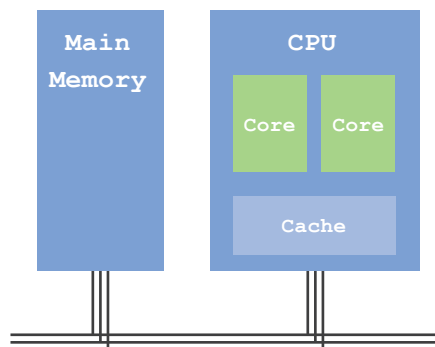
Execution time—ideally measured by actually running the program on a computer—can also be estimated using basic assumptions and an understanding of the steps the algorithm performs.

To keep our analysis realistic yet manageable, we categorize steps into two types:

1. Processor-only steps (like comparisons), which involve only the CPU cores and their internal registers.
2. Memory-related moves (like holds, shifts, and insertions), which involve moving data between memory locations, such as between cache and main memory (RAM).

In a real computer, memory access time can vary depending on whether data is stored in fast-access cache or slower main memory. However, since our

dataset is small, it's reasonable to assume that all memory-related steps occur within the processor's cache. Cache access is faster than accessing main memory, but still slower than internal CPU operations.



This figure is a simplified representation of the processor and memory components in a computer's architecture. The CPU contains multiple cores for executing instructions and a cache for fast, nearby access to frequently used data. The main memory (RAM) holds larger amounts of data for longer periods while programs run, but accessing it is slower than accessing the cache.

For the sake of estimation:

- We will assign 1 nanosecond (ns) to processor-only steps.
- We will assign 5 nanoseconds (ns) to memory-related steps, assuming cache-level latency.

This allows us to make performance comparisons based on an estimated execution time:

Notes

The time estimates used in these examples are intentionally simplified to support conceptual understanding. While actual timings depend on hardware architecture and system conditions, the goal is to help students recognize that different types of operations vary in cost—and that performance can be meaningfully estimated using reasonable assumptions.

- Each comparison = 1 ns
- Each hold, shift, or insert = 5 ns

By multiplying the number of each type of step by its estimated time cost, we can calculate the total execution time of the algorithm in nanoseconds.

This approach gives us a practical way to reason about performance, even without running the program on a real computer.

Index	Hold	Compare	Shift	Insert
1	1 (x 5)	1 (x 1)	1 (x 5)	1 (x 5)
2	1 (x 5)	2 (x 1)	2 (x 5)	1 (x 5)
3	1 (x 5)	3 (x 1)	3 (x 5)	1 (x 5)
4	1 (x 5)	4 (x 1)	4 (x 5)	1 (x 5)
Total	20 ns	10 ns	50 ns	20 ns

So for a 5-item dataset, under the worst-case scenario of a completely reversed list, the insertion sort program has an estimated execution time of 20 ns (holds) + 10 ns (comparisons) + 50 ns (shifts) + 20 ns (insertions) = 100 ns.

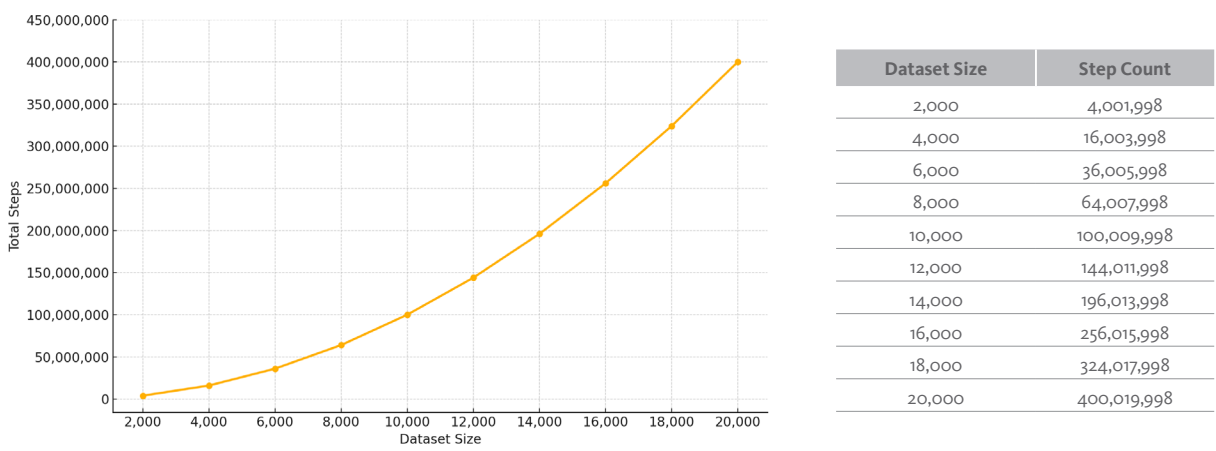
SCALABILITY

Now that we've explored how insertion sort performs on a small dataset, we can begin to explore what happens as the size of the dataset increases—a concept known as scalability.

As the dataset grows, both the step count and the execution time increase. This is especially true under the worst-case scenario, where the list is in completely reversed order.

As we scale up the size of the dataset, we can clearly see the impact on step count. With every new item added, the algorithm must compare it with all preceding items and shift them if necessary. This means that each additional item increases the total number of steps more than the one before it.

If we run simulations on datasets of increasing size—starting from 2,000 and going up to 20,000 items—we can demonstrate that, under the worst-case scenario, the step count grows rapidly. In fact, the amount of increase itself becomes greater with each step up in dataset size. This shows that the algorithm does not scale well and performs poorly as the size of the dataset increases.



This table and graph show how the total number of steps required by the insertion sort algorithm increases as the dataset size grows from 2,000 to 20,000 items under the worst-case scenario. The steep upward curve illustrates how step count accelerates with each additional item, highlighting the algorithm’s poor scalability for large datasets.

Notes

This is an example of quadratic growth. The total step count increases approximately with the square of the dataset size, which is why performance drops rapidly as the dataset grows.

Now let's examine how step count reflects on execution time. One might assume that execution time increases in the same way step count increases as the dataset grows—and while that's generally true, execution time behaves slightly differently. That's because it depends not only on the number of steps or operations, but also on how long each operation takes to execute.

The time required to execute an operation depends on where the data is stored. The closer the data is to the processor, the faster it can be accessed. Small datasets usually fit entirely in the CPU's cache, where access is very fast. But with large datasets, the processor may need to access main memory (RAM), which is much slower.

This introduces memory latency—the time it takes to fetch or store data from memory—which becomes a noticeable part of total execution time for large datasets.

EXECUTION TIME DEPENDS ON BOTH STEP COUNT AND MEMORY LATENCY.

Going back to our simulation, let's determine the memory latency for each operation. As the dataset increases in size, more memory is needed to hold it. This means the data is likely to be held further from the processor, moving from fast-access cache to slower main memory (RAM).

Because of this, we assign different latency values depending on the dataset size:

- For small datasets, the data is likely held in the CPU's cache, where access is very fast.
- For medium-sized datasets, the data may spill into L3 cache, which is slower.
- For larger datasets, the data is held in main memory (RAM), where access time is significantly longer.

Comparisons always occur within the CPU's registers and are very fast (typically around 1 nanosecond or less).

Memory-related moves—including holds, shifts, and insertions—require access to memory beyond the CPU and are more sensitive to dataset size and memory latency.

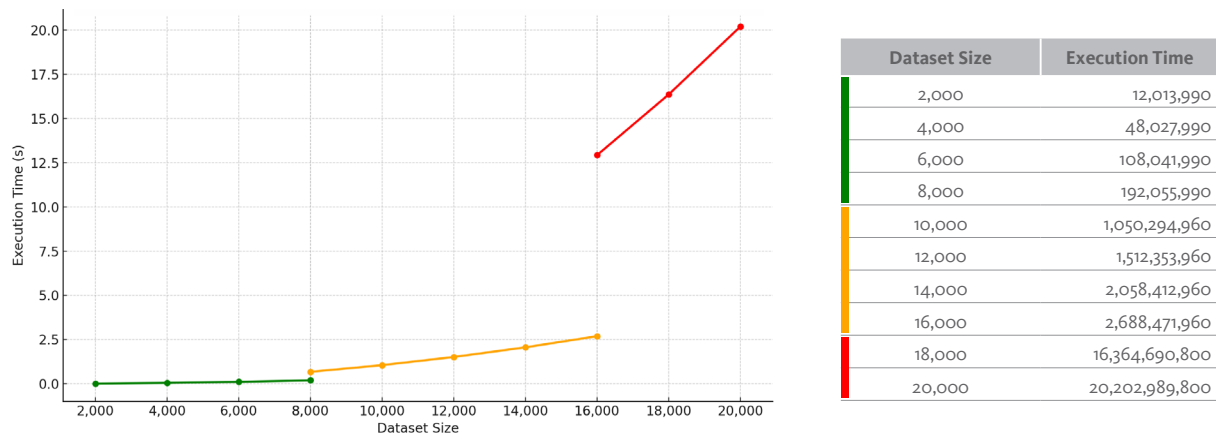
Dataset Size	Memory Location	Comparison (CPU)	Hold / Shift / Insert (Memory)
2,000–8,000	L1/L2 Cache	1 ns	5 ns
8,001–16,000	L3 Cache	1 ns	20 ns
16,001–20,000	Main Memory	1 ns	100 ns

According to these estimates, we can calculate execution times for each dataset size. As the dataset grows, it gradually begins spilling into higher memory tiers. Smaller datasets are held close to the processor in the L1 and L2 caches, but as the size exceeds the capacity of these caches, the data begins to spill into the L3 cache—which, in our simplified simulation, occurs at around 8,000 items. For even larger datasets that exceed the capacity of the L3 cache (around 16,000 items in the simulation), the data must be held in main memory (RAM).

Notes

In real systems, memory access doesn't shift abruptly between tiers. Instead, as the dataset grows, portions of it gradually move from fast-access cache to slower memory. Our simulation simplifies this behavior by assigning a dominant memory tier based on dataset size. This helps students understand the concept of memory latency without needing to model the complexities of real-world caching systems.

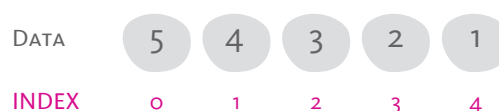
As a result, the algorithm slows down not only due to the increasing number of steps, but also due to memory latency—the added time required to access data stored in slower memory tiers.



This table and graph show how execution time increases as dataset size grows from 2,000 to 20,000 items using the insertion sort algorithm under the worst-case scenario. Execution time is shown in nanoseconds in the table and converted to seconds in the graph. The curve is divided into three color-coded segments based on the type of memory used—cache (green), L3 cache (orange), and main memory (red). The sharp rise in execution time highlights the combined effect of step count growth and increased memory latency, revealing the algorithm's poor scalability with larger datasets.

BUBBLE SORT

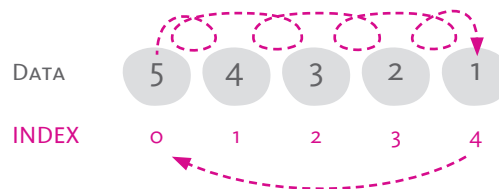
Let's explore the performance of the bubble sort algorithm in the same way we did for insertion sort. We'll start with a small list of 5 numbers that need to be sorted in ascending order. Since we're interested in the worst-case scenario, the list will begin in descending order—meaning that every number is in the opposite position of where it should be.



STEP COUNT

In the same way we examined step count for the insertion sort algorithm, we'll group the steps into two categories: comparisons and moves. The difference is that instead of holds, shifts, or insertions, bubble sort uses swaps to move data.

The bubble sort algorithm repeatedly compares adjacent pairs of data items and swaps them if they are in the wrong order. This process is controlled by the outer loop, which runs a fixed number of times regardless of whether the list becomes sorted earlier. For a list with 5 items, the outer loop runs 4 times, and during each pass, the inner loop performs 4 comparisons.



This diagram shows the structure of a single pass in the bubble sort algorithm. Each curved arrow represents a comparison between adjacent values. In the worst-case scenario, every comparison results in a swap. While the number of comparisons remains the same in every pass, the number of swaps decreases as larger values are moved to the end of the list.

Each pass in the bubble sort algorithm makes 4 comparisons (for a 5-item list), regardless of whether the list becomes partially sorted. The pattern of comparing adjacent pairs remains the same across all passes. However, the number of swaps decreases with each pass because the largest remaining value is moved into its correct position at the end of each round.

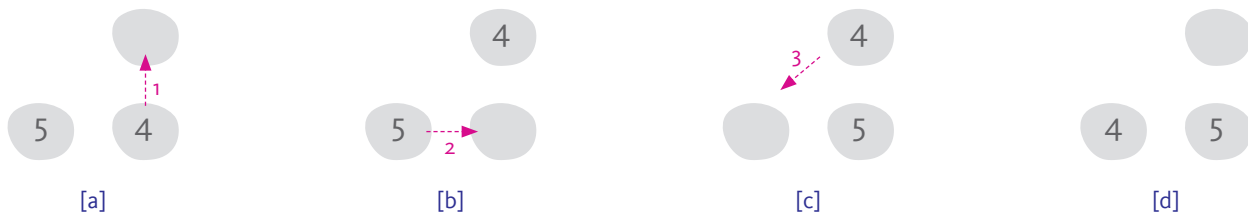
- In Pass 1, there are 4 comparisons and 4 swaps.
- In Pass 2, there are 4 comparisons and 3 swaps.
- In Pass 3, there are 4 comparisons and 2 swaps.
- In Pass 4, there are 4 comparisons and 1 swap.

So for a 5-item dataset, under the worst-case scenario of a completely reversed list, the bubble sort algorithm requires a total step count of 16 (comparisons) + 10 (swaps) = 26 steps.

Pass	Compare	Swaps
1	4	4
2	4	3
3	4	2
4	4	1
Total	16	10

However, it is important to examine the swap in detail as a type of move. A swap is not simply a single move of data from one place in memory to another; it actually requires three separate moves.

In real memory operations, one data item cannot simply overwrite a location already occupied by another. Instead, the existing value must first be moved to an empty temporary location, the target location is then overwritten with the new value, and finally, the temporarily held value is written into the second location. This sequence means that a single swap involves three distinct moves.



This figure shows the three moves involved in a swap operation during bubble sort. [a] The value 4 is moved to a temporary location. [b] The value 5 takes the place of 4. [c] The stored value 4 is moved from the temporary location into 5's original position. [d] The swap is complete, with 4 and 5 now in their new positions.

Accordingly, we must revise our step count to account for three moves per swap. This gives a total step count of 16 (comparisons) + 30 (moves) = 46 steps.

Pass	Compare	Swaps	Moves
1	4	4	12
2	4	3	9
3	4	2	6
4	4	1	3
Total	16	10	30

EXECUTION TIME

Having detailed the steps required for the bubble sort algorithm to sort a worst-case scenario list of 5 items, we can now discuss the time it would take to execute such a task. Using the same estimates of time as before—comparisons at 1 ns each and moves at 5 ns each—we can calculate the execution time.

- Comparisons: $16 \times 1 \text{ ns} = 16 \text{ ns}$
- Moves: $30 \times 5 \text{ ns} = 150 \text{ ns}$

This gives a total execution time of: $16 \text{ ns} + 150 \text{ ns} = 166 \text{ ns}$.

When we compare the results for a worst-case scenario list of 5 items, we see a clear difference between the two algorithms. Insertion sort required 28 steps and 100 ns to complete the task, while bubble sort required 46 steps and 166 ns.

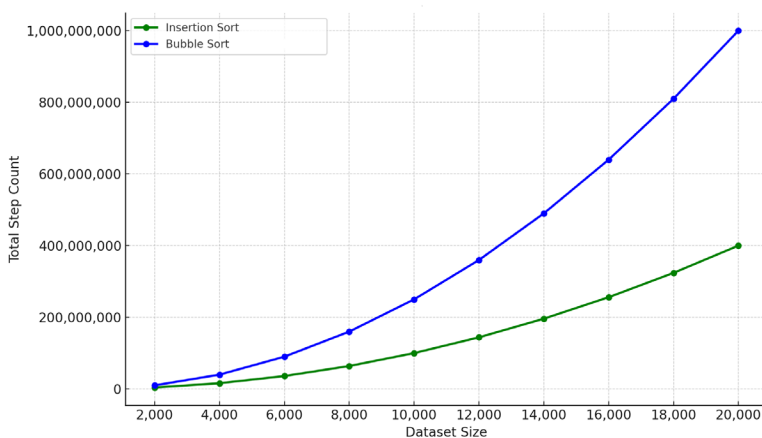
This means that bubble sort performed more operations and took longer to execute for the same dataset and conditions. Even at this small scale, the difference highlights bubble sort's lower efficiency compared to insertion sort in both step count and execution time.

SCALABILITY

While the difference in performance is already visible with a small dataset, it becomes even more pronounced as the dataset grows. Because bubble sort makes a fixed number of comparisons in every pass and performs swaps that require multiple moves, the total step count increases rapidly with larger lists.

As we saw with insertion sort, more steps mean more time—but for bubble sort, the higher number of moves per swap amplifies this effect. To understand the full impact, we need to examine how bubble sort scales with dataset size and how this affects execution time.

Let's return to our simulation using dataset sizes ranging from 2,000 to 20,000 data items. We will begin by comparing the insertion sort algorithm with the bubble sort algorithm, focusing on the scalability of their step counts.



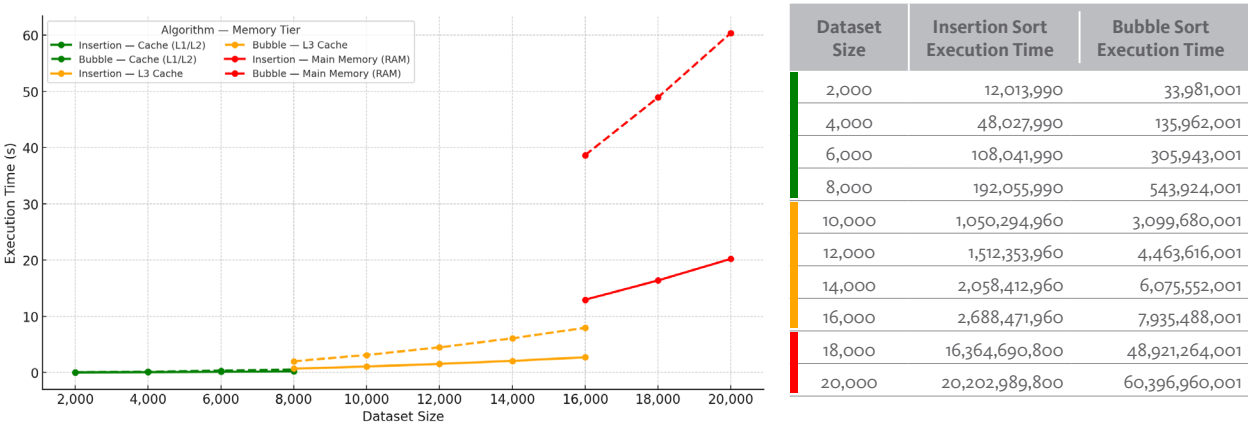
Dataset Size	Insertion Sort Step Count	Bubble Sort Step Count
2,000	4,001,998	9,993,001
4,000	16,003,998	39,986,001
6,000	36,005,998	89,979,001
8,000	64,007,998	159,972,001
10,000	100,009,998	249,965,001
12,000	144,011,998	359,958,001
14,000	196,013,998	489,951,001
16,000	256,015,998	639,944,001
18,000	324,017,998	809,937,001
20,000	400,019,998	999,930,001

This graph and table compare the total step counts for insertion sort and bubble sort under the worst-case scenario, for dataset sizes ranging from 2,000 to 20,000 items. Both algorithms exhibit quadratic growth, but bubble sort consistently requires about 2.5 times more steps. The gap between the two curves widens as dataset size increases, clearly illustrating bubble sort's poorer scalability.

Examining the graph and table, it becomes clear that bubble sort is far less efficient than insertion sort. The main reason is that bubble sort’s structure forces it to perform a fixed number of comparisons in each pass, regardless of how much of the list is already sorted.

In a worst-case scenario, every comparison results in a swap, and each swap involves three separate moves in memory. This extra work adds up quickly, making bubble sort require about 2.5 times more steps than insertion sort for the same dataset size. As the dataset grows, this difference becomes increasingly significant, highlighting how the mechanics of an algorithm can have a major impact on its performance.

These differences become even more pronounced when we calculate the execution time required for each dataset. As the dataset grows, it often exceeds the capacity of faster memory tiers (such as the CPU’s cache) and must be stored in slower tiers (such as L3 cache or main memory). This increase in memory latency further slows the execution of the sorting process, amplifying the performance gap between the two algorithms.



This graph and table compare execution times for insertion sort and bubble sort under the worst-case scenario for dataset sizes from 2,000 to 20,000 items. Execution time is shown in nanoseconds in the table and converted to seconds in the graph. Times are segmented by memory tier—cache (green), L3 cache (orange), and main memory (red). Bubble sort’s higher step count and costly swaps cause its execution time to grow much faster, especially as larger datasets move into slower memory.

Understanding algorithm performance goes beyond knowing whether an algorithm produces the correct result. How efficiently it solves a problem—measured in step count, execution time, and memory use—can make a significant difference, especially with large datasets or time-sensitive tasks.

As seen in our comparison of insertion sort and bubble sort, two algorithms with the same growth rate can perform very differently in practice due to their mechanics and the cost of individual operations. Choosing the right algorithm is therefore about matching the method to the problem, the dataset size, and the available computing resources.

SUMMARY

Sorting – the arrangement of data in a specific order based on a certain criterion.

- Order is the direction—ascending or descending.
- Criterion is what we sort by, such as exam scores, prices, names, or titles.
- Sorting helps organize data, making it easier to search, interpret, and analyze.

Algorithm & Program –

- An algorithm is a series of steps designed to perform a particular task.
- A program is the set of written instructions a computer can execute to run an algorithm.
- A variable is a label whose value or content can change. Variables in programming need to be declared, meaning their label is specified and an initial value is assigned.
- Flow is the sequence in which the steps of a program or algorithm are carried out.
- Control refers to the conditions that determine this sequence, such as when certain steps are executed, repeated, or skipped.
- Loops are a common control structure used to repeat a set of instructions, while conditional statements decide which instructions run based on specific criteria.

- A **block** is a segment of code that performs a particular task. A program is typically organized into three main blocks: initialization, main body, and termination. Blocks can be **sequential** (one after another) or **nested** (one inside another).
- Indentation is used to show which instructions belong to a loop block.

Insertion Sort – Builds a sorted section one item at a time.

- Outer Loop – Moves through each index in the list (starting at the second item) and hands control to the inner loop.
- Inner Loop – Compares the held item with preceding items, shifts items as needed, and inserts the held item into its correct position based on the sorting criterion.
- Strategy – Compare, shift, insert.

Bubble Sort – Repeatedly passes through the list, swapping adjacent items if out of order.

- Outer Loop – Repeats the inner loop for a fixed number of passes to ensure the list becomes fully sorted.
- Inner Loop – Compares adjacent pairs and swaps them if they are out of order, gradually moving items into their correct positions based on the sorting criterion.
- Strategy – Pair, compare, swap.

Performance –

- Step count and execution time are two key measures of performance.
- Worst-Case Scenario – The most challenging dataset arrangement for an algorithm (e.g., completely reversed list for ascending sort). Used for fair comparisons.
- Scalability refers to how an algorithm's performance changes as the dataset size increases.
- Memory latency is the delay between requesting data from memory and the moment it becomes available for use.
- Insertion sort is a more efficient algorithm than bubble sort, requiring fewer steps and less execution time. The difference widens as datasets grow, especially when memory latency becomes significant.

Activity 4.1

Read the following statements, and write on the line provided for each statement 'T' if it is true and 'F' if it is false.

T/F	STATEMENT
<u>T</u>	Sorting is the arrangement of data in a specific order based on a certain criterion.
<u>F</u>	In programming, a variable is a label whose value cannot change.
<u>F</u>	The order in sorting refers to the basis or criterion used to determine how data is arranged.
<u>T</u>	Flow refers to the sequence of steps in a program, while control refers to the conditions that decide which steps are executed, repeated, or skipped.
<u>T</u>	A program is typically organized into initialization, main body, and termination blocks.
<u>T</u>	In insertion sort, the inner loop compares the held item with preceding items, shifts items as needed, and inserts it in the correct position.
<u>F</u>	In bubble sort, each swap requires one move in memory.
<u>T</u>	Step count and execution time are two measures used to compare algorithm performance.
<u>F</u>	A worst-case scenario is the easiest dataset arrangement for an algorithm to sort.
<u>T</u>	As dataset size increases, the need for slower memory tiers can raise memory latency and increase execution time.

Activity 4.2

Answer the questions below.

1. What does “order” refer to in sorting?
 - a) The basis for sorting
 - b) The sequence in which algorithms are written
 - c) **The direction of arrangement**
 - d) The type of data being sorted

2. Which of the following is an example of a numeric sorting criterion?
 - a) Student names
 - b) **Exam scores**
 - c) Book titles
 - d) File names

3. Which statement about sorting criteria is correct?
 - a) They can only be numeric
 - b) **They can be numeric or alphanumeric**
 - c) They must be alphabetical
 - d) They must be in descending order

4. Which of the following is an example of alphanumeric data?
 - a) Temperatures
 - b) Prices
 - c) **ID codes**
 - d) Time

5. Why is sorting used?
 - a) To make programs run faster
 - b) **To organize data for easier search and analysis**
 - c) To reduce memory usage
 - d) To prevent data duplication

Activity 4.3

Answer the questions below.

1. What is an algorithm?
 - a) A program written in a specific language
 - b) A series of steps designed to perform a task
 - c) A list of variables
 - d) A method for compiling code
2. What is a program?
 - a) A set of instructions a computer can execute
 - b) A flowchart showing an algorithm
 - c) A memory location in a computer
 - d) A sorting method
3. What does declaring a variable involve?
 - a) Stating its label and assigning an initial value
 - b) Sorting its contents
 - c) Creating a loop for it
 - d) Printing it to the screen
4. Which of the following controls the order in which program steps run?
 - a) Sorting
 - b) Variables
 - c) Flow & control structures
 - d) Program termination
5. Which is NOT a typical program block?
 - a) Initialization
 - b) Main body
 - c) Termination
 - d) Compilation

Activity 4.4

Answer the questions below.

1. What is the basic strategy of insertion sort?
 - a) Pair, compare, swap
 - b) **Compare, shift, insert**
 - c) Merge, split, compare
 - d) Select, swap, compare

2. What does the outer loop in insertion sort do?
 - a) Compares adjacent pairs
 - b) Holds an item and inserts it in its correct position
 - c) **Moves through each index and hands control to the inner loop**
 - d) Swaps items until sorted

3. What does the inner loop in insertion sort do?
 - a) Pairs adjacent items for comparison
 - b) **Shifts items and inserts the held item in the correct position**
 - c) Selects the smallest value in the list
 - d) Divides the list into halves

4. Which step is NOT part of insertion sort?
 - a) Compare
 - b) Shift
 - c) Insert
 - d) **Swap**

5. In the worst-case scenario, how is the dataset arranged for insertion sort?
 - a) Already in the intended order
 - b) **In reverse of the intended order**
 - c) In random order
 - d) Grouped by type

Activity 4.5

Answer the questions below.

1. What is the basic strategy of bubble sort?
 - a) Compare, shift, insert
 - b) Merge, compare, shift
 - c) **Pair, compare, swap**
 - d) Select, swap, compare

2. What does the outer loop in bubble sort do?
 - a) Moves through the list starting at the second item
 - b) Compares each item with all previous items
 - c) **Repeats the inner loop until a pass completes with no swaps**
 - d) Inserts items into the sorted section

3. What does the inner loop in bubble sort do?
 - a) Shifts items forward
 - b) **Swaps adjacent items if out of order**
 - c) Holds an item for insertion
 - d) Divides the list in half

4. How many moves are required for one swap in bubble sort?
 - a) 1
 - b) 2
 - c) **3**
 - d) 4

5. In bubble sort, after each pass, which item is moved into its correct position?
 - a) The largest item
 - b) The smallest item
 - c) **The next item in the intended order**
 - d) The item with the highest index

Activity 4.6

Answer the questions below.

1. Which two measures are common for evaluating algorithm performance?
 - a) Order and criterion
 - b) Memory and variables
 - c) Flow and control
 - d) **Step count and execution time**

2. What is a worst-case scenario?
 - a) A program without loops
 - b) The least challenging arrangement of data
 - c) **The most challenging arrangement of data**
 - d) A dataset with missing values

3. What does scalability describe?
 - a) How well an algorithm works on small datasets
 - b) **How performance changes as dataset size increases**
 - c) How memory is allocated in a program
 - d) How variables are declared

4. What is memory latency?
 - a) The speed of the processor
 - b) **The delay between requesting data from memory and receiving it**
 - c) The time taken to write a program
 - d) The order in which data is stored

5. Why is insertion sort generally more efficient than bubble sort?
 - a) It requires more swaps
 - b) It works only on small datasets
 - c) **It requires fewer steps and less execution time**
 - d) It avoids using loops

Activity 4.7

In this activity, you will use a spreadsheet application to sort three different lists of numbers. You will then extract specific information from each list before sorting and after sorting, and reflect on the time and effort it took.

Download the spreadsheet here: bit.ly/Activity4_7

- a. List 1 – 50 Random 6-Digit Numbers (100,000–999,999)

The smallest number: 102372

The largest number: 955156

The duplicate number: 312944

Find the number specified in the spreadsheet.

- b. List 2 – 100 Random 6-Digit Numbers (100,000–999,999)

The smallest number: 102372

The largest number: 955156

The duplicate number: 704924

Find the number specified in the spreadsheet.

Did the larger dataset size make it easier or harder to find the data without sorting? Explain:

It harder to find values without sorting because there is more data to scan.

Sorting makes locating values much faster.

- c. List 3 – 100 Random 6-Digit Numbers (222,222–333,333)

The smallest number: 222354

The largest number: 331588

The duplicate number: 319738

Find the number specified in the spreadsheet.

Did the larger dataset size make it easier or harder to find the data without sorting? Explain:

Numbers look more alike, making numbers harder to spot.

Sorting makes locating values much faster.

Activity 4.8

In this activity, you will work with data presented in table form, each representing a real-world scenario. Sort the data and extract the information needed to answer the questions. Use sorting and functions such as COUNT and AVERAGE to investigate and answer.

Download the spreadsheet here: bit.ly/Activity4_8

- a. You are given an excerpt from a school database for Grades 7 and 8 with the following fields: Grade, Section, Student ID, and GPA.

1. How many students are in each grade?

Grade 7 — 32 ; Grade 8 — 35

2. If the school wants to recognize the top 3 students in each grade, who are they? List their student IDs.

Grade 7 — ID-2885, ID-2696, ID-7516

Grade 8 — ID-7309, ID-1489, ID-2598

3. If the school is offering support to students with a GPA below 2.00, how many students in each grade would qualify?

Grade 7 — 8 students

Grade 8 — 7 students

4. Looking at the number of students in each section, which section in each grade has the fewest students and which has the most?

Grade 7 — fewest A (9), most B (12)

Grade 8 — fewest B (10), most C (13)

5. If the school wants to recognize the teacher whose section has the highest average GPA, which section would win? Show the average GPA for each section to support your answer.

Grade 7 — A: 2.65, B: 2.60, C: 2.76 (highest)

Grade 8 — A: 2.59, B: 3.35 (highest), C: 2.88

- b. You are given an excerpt from a logistics company's delivery database showing: Order Number, Route, Day of Delivery, and Time En Route (minutes).

1. Which order was delivered the fastest, and which was the slowest?

Fastest: 700182 (29 min) ; Slowest: 700125 (62 min)

2. For each delivery day, which order was delivered the fastest?

Monday: 700115 (35 min) ; Wednesday: 700139 (35 min) ; Friday: 700182 (29 min)

3. Deliveries that require more than 50 minutes are considered slow. Compare delivery days and routes by the number of slow deliveries. What stands out?

By day — Monday: 15, Wednesday: 6, Friday: 5

By route — Route 1: 7, Route 2: 4, Route 3: 15

Monday and Route 3 have the most slow deliveries.

4. Which route has the lowest average delivery time? Show the average time for each route to support your answer.

Route 2 — 44.10 min (best) ; Route 1 — 45.63 min ; Route 3 — 47.83 min

5. For each delivery day, which route has the lowest average delivery time? Recommend the best route to take on each day and support your answer with the average times.

Monday: Route 2 — 38.10 min

Wednesday: Route 1 — 38.90 min

Friday: Route 3 — 38.30 min

Notes

Use the tables in this lesson to explain that databases store information in tables, with fields (column headings) and entries (rows of data). Point out that sorting, counting, and averaging are basic operations used in database queries, laying the groundwork for more formal database concepts in later lessons.

Activity 4.9

Use the Insertion Sort algorithm to arrange the two given lists in ascending order. For each step, record the number of comparisons, shifts, and inserts you perform, and write the updated list. Keep a running total of the comparisons, shifts, inserts, and total operations at the bottom of the table.

a. [C, D, A, B]

Index	Hold	Compare	Shift	Insert	List
1	1	1	0	0	C, D, A, B
2	1	2	2	1	A, C, D, B
3	1	3	2	1	A, B, C, D
Total	3	6	4	2	15

b. [1, 3, 0, 2, 4, 5]

Index	Hold	Compare	Shift	Insert	List
1	1	1	0	0	1, 3, 0, 2, 4, 5
2	1	2	2	1	0, 1, 3, 2, 4, 5
3	1	2	1	1	0, 1, 2, 3, 4, 5
4	1	1	0	0	0, 1, 2, 3, 4, 5
5	1	1	0	0	0, 1, 2, 3, 4, 5
Total	5	7	3	2	17

Activity 4.10

Use the Bubble Sort algorithm to arrange the two given lists in ascending order. For each pass through the list, record the number of comparisons and swaps you perform, and write the updated list. Keep a running total of the comparisons, swaps, and total operations at the bottom of the table.

a. [B, C, D, A]

Pass	Compare	Swap	List
1	3	1	B, C, A, D
2	3	1	B, A, C, D
3	3	1	A, B, C, D
Total	9	3	12

b. [2, 4, 0, 3, 5, 1]

Pass	Compare	Swap	List
1	5	3	2, 0, 3, 4, 1, 5
2	5	2	0, 2, 3, 1, 4, 5
3	5	1	0, 2, 1, 3, 4, 5
4	5	1	0, 1, 2, 3, 4, 5
5	5	0	0, 1, 2, 3, 4, 5
Total	25	7	32

Activity 4.11

Review the pseudocode program for Insertion Sort. Then answer the comprehension questions provided.

```
START InsertionSort
OPEN list
SET hold_index (1)
SET end_index (last index of list)

REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    AS_LONG_AS hold_data > compare_data
        SHIFT compare_data FROM compare_index TO compare_index + 1
        RESET compare_index (compare_index - 1)
        RESET compare_data (data_item AT compare_index)

    INSERT hold_data TO compare_index
    RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END InsertionSort
```

1. Which two variables are declared in the initialization block?

hold_index, end_index

2. What does this instruction do: RESET compare_index (compare_index - 1)?

This instruction moves the comparison one position to the left.

3. What does this instruction do: REPEAT_UNTIL hold_index > end_index?

Ensures the program ends when it reaches the end of the list.

4. Which instruction moves the sorting to the next index in the list?

RESET hold_index (hold_index + 1)

5. Which instruction would you change to sort the list in descending order?

Change the instruction to: AS_LONG_AS hold_data < compare_data

Activity 4.12

Review the pseudocode program for Bubble Sort. Then answer the comprehension questions provided.

```
START BubbleSort
OPEN list
SET end_index (last index of list)
SET swap_count (1)

REPEAT_UNTIL swap_count (0)
  RESET swap_count (0)
  SET hold_index (1)

  REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    IF compare_data > hold_data THEN
      SWAP_POSITIONS (compare_data, hold_data)
      RESET swap_count (swap_count + 1)

    RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END BubbleSort
```

1. Which two variables are declared in the initialization block?

end_index, swap_count

2. What does this instruction do: REPEAT_UNTIL swap_count (0)

Repeats passes through the list until no swaps occur.

3. Which instruction moves the sorting to the next index in the list?

RESET hold_index (hold_index + 1)

4. What happens in RESET swap_count (swap_count + 1)?

It increases the swap counter by one each time a swap is made.

5. Which instruction would you change to sort the list in descending order?

Change the condition to IF compare_data < hold_data THEN.

Activity 4.13

Having learned the mechanics, performance, and logic behind insertion and bubble sort algorithms, it's time to explore them further. This activity allows students to examine these algorithms in action using provided Python programs. Follow the links to download the three programs for this activity or use the short link: bit.ly/Activity4_13.

1. [Insertion Sort Program](#): Creates a list from all integers in a user-defined range (maximum 20 items). The list can be generated in worst-case (descending) or random order. The program shows the list after each completed insert (labeled by index) and reports totals for comparisons, shifts, and inserts.
2. [Bubble Sort Program](#): Builds the same kind of list (worst-case or random). The program shows the list after each completed pass (labeled by pass number) and reports totals for comparisons and swaps.
3. [Combined Sorting Program](#): This program includes both insertion and bubble sort. It generates a dataset from all integers in a user-defined range and runs both algorithms on the same list.
 - If the list size is 10,000 items or fewer, the program runs both algorithms and reports operation counts and execution time.
 - If the list size is larger than 10,000, the program offers a Theory Mode that displays the worst-case (reverse-sorted) operation counts without running the algorithms.

Notes

Notice the similarity between the pseudocode and the Python programs provided for insertion and bubble sort. This shows how pseudocode serves as a bridge between algorithmic logic and actual coding. By following the same variable names and structure, students can clearly see how the logic of each step is translated into Python. Encourage students to use pseudocode as a guide to understand sorting processes, compare algorithm performance, and develop their programming skills.

